Laboratory of Computational Fluid Dynamics

Exercise Report #01

Numerical Integration of the Logistic Equation

Academic Year: 2025/2026
Department: Department of Industrial Engineering
Institution: University of Padova

Student Information

Name: Bordin Gianmarco
Student ID: 215 76 00
Email: gianmarco.bordin@studenti.unipd.it

Contents

1 Introduction

The logistic equation is a fundamental non-linear Ordinary Differential Equation (ODE) used to describe population dynamics under resource constraints. The model transitions from an initial exponential growth to a saturation regime governed by a carrying capacity. The Cauchy problem considered in this work is:

$$\begin{cases} \frac{d\phi(t)}{dt} = r\phi(t)\left(1 - \frac{\phi(t)}{K}\right) \\ \phi(0) = \phi_0 = 0.5 \end{cases} \tag{1}$$

where $r = 1$ is the intrinsic growth rate and $K = 10$ is the carrying capacity. The analytical solution is known and given by:

$$\phi(t) = \frac{K}{1 + \frac{K-\phi_0}{\phi_0} e^{-rt}} \tag{2}$$

The integration is performed over the time interval $t \in [0, 10]$ using five different time steps: $\Delta t \in \{1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$. The numerical integration is performed using two implicit methods: the Implicit Euler method (first-order accurate) and the Crank-Nicolson method (second-order accurate).

Because both integration schemes are implicit, the numerical computation at each time step requires the solution of a non-linear algebraic equation of the form $g(\phi_{n+1}, \phi_n) = 0$. This root-finding problem is handled using the iterative Newton-Raphson algorithm.

2 Numerical Implementation

The numerical solver is developed in Modern Fortran using a highly modular, object-oriented-like architecture. The software is organized into specific modules to separate the physical model, the mathematical tools, and the time-integration engines:

- `mod_param`: Defines the floating-point precision, allowing a seamless switch between double (`real64`) and single (`real32`) precision via a single parameter `rp`. It also defines abstract interfaces for function passing.
- `mod_equations`: Contains the physical parameters (r, K) and implements the Right-Hand Side (RHS) of the ODE $f(t, \phi)$, its analytical derivative with respect to ϕ ($\frac{\partial f}{\partial \phi}$), and the exact analytical solution.
- `mod_math`: Isolates the Newton-Raphson root-finding algorithm. It is designed to accept generic residual functions and their Jacobians via procedure pointers. Convergence is checked dynamically against a mixed absolute-relative tolerance based on the machine epsilon ($100 \times \epsilon$). Additional safeguards are included to detect zero-derivative conditions and prevent silent propagation of floating-point exceptions.
- `mod_solver`: Contains the two time-integration subroutines (`compute_imp_euler_nonlin` and `compute_crank_nic_nonlin`). Inside these routines, the specific residual functions $g(\phi_{n+1})$ and their Jacobians $g'(\phi_{n+1})$ are formulated and passed to the Newton-Raphson solver.

For the Implicit Euler scheme, the residual and Jacobian are:

$$g(\phi_{n+1}) = \phi_{n+1} - \phi_n - \Delta t f(\phi_{n+1}) \tag{3}$$

$$g'(\phi_{n+1}) = 1 - \Delta t \frac{\partial f}{\partial \phi}(\phi_{n+1}) \tag{4}$$

For the Crank-Nicolson scheme, they are defined as:

$$g(\phi_{n+1}) = \phi_{n+1} - \phi_n - \frac{\Delta t}{2}[f(\phi_{n+1}) + f(\phi_n)] \tag{5}$$

$$g'(\phi_{n+1}) = 1 - \frac{\Delta t}{2}\frac{\partial f}{\partial \phi}(\phi_{n+1}) \tag{6}$$

3 Results and Discussion

3.1 Time Evolution (Task 1)

Figure 1 shows the numerical solutions obtained with $\Delta t = 1$ alongside the exact solution.

As observed in the plot, both numerical schemes successfully capture the overall S-shaped trend of the logistic curve and eventually converge to the correct carrying capacity ($K = 10$) at steady state. However, the transient phase highlights a significant difference in accuracy, which is exacerbated by the exceptionally coarse time step ($\Delta t = 1$).

The first-order Implicit Euler method visibly overestimates the growth rate during the initial exponential phase, resulting in a pronounced deviation from the exact analytical curve. This behavior is typical of its $\mathcal{O}(\Delta t)$ truncation error, which introduces substantial inaccuracy when the time step is large. Conversely, the second-order Crank-Nicolson scheme matches the exact solution almost perfectly throughout the entire integration domain. This visual comparison clearly demonstrates how the higher-order temporal discretization of the Crank-Nicolson method ($\mathcal{O}(\Delta t^2)$) yields highly accurate predictions even on very coarse temporal grids, successfully minimizing the discretization error during the rapid transient evolution.

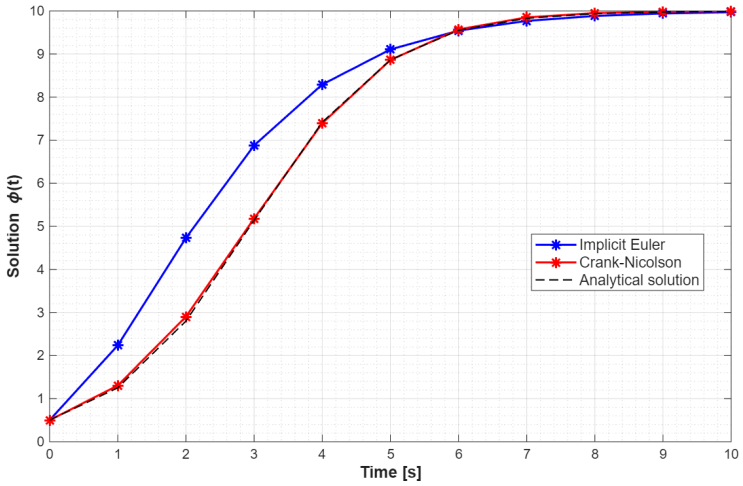

Figure 1: Comparison of Implicit Euler and Crank-Nicolson methods vs. exact solution for $\Delta t = 1$.

3.2 Convergence Analysis in Double Precision (Task 2)

To verify the formal accuracy of the schemes, the absolute error $|e_N| = |\phi_{num}(T) - \phi_{exact}(T)|$ at the final time $T = 10$ is evaluated for progressively smaller time steps. Computations are performed in double precision (`real64`).

As shown in Figure 2, the error curves in the log-log plot follow straight lines, indicating a power-law convergence. A fitting procedure of the form $|e_N| = A\Delta t^p$ (enforcing $p = 1$ for Euler and $p = 2$ for Crank-Nicolson, without constant offsets) confirms the theoretical expectations: the Implicit Euler method reduces the error linearly ($O(\Delta t)$), while the Crank-Nicolson method exhibits a quadratic reduction ($O(\Delta t^2)$).

Specifically, the regression yields prefactors of $A \approx 0.021$ for the Implicit Euler scheme and $A \approx 0.003$ for the Crank-Nicolson scheme, strictly validating the formal order of accuracy. To rigorously determine the prefactors A, a least-squares regression was performed directly on the numerical data. Following the theoretical constraints, the regressions were forced through the origin to prevent the introduction of unphysical constant offsets (i.e., strictly enforcing $|e_N| = A\Delta t^p$). Computationally, this was implemented in MATLAB by solving the overdetermined systems analytically via the left-matrix divide operator (`\`). By executing the operations `dt \ err_euler` and `(dt.^2) \ err_cn`, the algorithm minimized the sum of the squared residuals while guaranteeing a zero-intercept

fit, fully respecting the formal truncation error definitions.

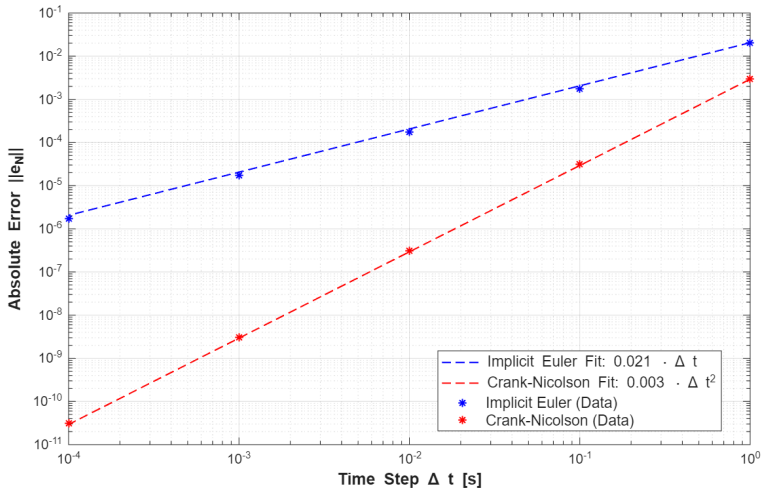

Figure 2: Absolute error at $T = 10$ vs Δt in double precision (log-log scale) with theoretical fits.

3.3 Precision Limits: Single vs. Double Precision (Tasks 3 & 4)

To address Task 4, we first outline the fundamental difference between double and single precision in floating-point arithmetic. Double precision (`real64`) utilizes 64 bits to represent a number, granting approximately 15–17 significant decimal digits and a machine epsilon $\epsilon \approx 10^{-16}$. Single precision (`real32`), on the other hand, is limited to 32 bits, reducing the accuracy to roughly 7 significant digits with a machine epsilon $\epsilon \approx 10^{-7}$. This drastic reduction in available digits physically restricts the computer's ability to store infinitesimally small numbers, directly impacting the accumulation of round-off errors during long numerical integrations.

In accordance with Task 3, the convergence analysis was repeated using single precision (Figure 3). As requested, the theoretical curve fits were omitted since the standard power-law convergence is no longer indefinitely sustained.

In the single precision case, both error curves eventually deviate from their theoretical slopes and exhibit a distinctive V-shaped behavior. This phenomenon represents the classic competition between the discretization error and the machine round-off error, which can be formalized through the following estimate of the total global error:

$$E_{\text{tot}} \approx A \cdot \Delta t^{\,p} + \frac{B}{\Delta t}, \tag{7}$$

where the first term represents the truncation error (decreasing with Δt, with $p = 1$ or $p = 2$ depending on the scheme) and the second term accounts for the accumulated round-off noise, which grows as Δt decreases due to the increasing number of time steps $N = T/\Delta t$. The constant B is proportional to the machine epsilon ε and to the residual error introduced at each Newton-Raphson iteration. This two-term model predicts the existence of an optimal time step Δt^* that minimizes the total error, beyond which further refinement is counterproductive. For the Crank-Nicolson scheme in single precision, this optimum is reached around $\Delta t \approx 10^{-2}$, consistently with the observed V-shaped behavior in Figure 3.

For the second-order Crank-Nicolson scheme, the mathematical truncation error drops rapidly. However, the local convergence of the non-linear solver is restricted by the single-precision Newton-Raphson tolerance (tol $= 100 \times \epsilon \approx 10^{-5}$). Once the discretization error falls below this threshold (around $\Delta t = 10^{-2}$), the solver's local residual acts as continuous numerical noise. Over thousands of time steps, these microscopic round-off errors accumulate, completely overwhelming the vanishing discretization error and driving the global error sharply upward at $\Delta t = 10^{-3}$ and 10^{-4}.

Conversely, the first-order Implicit Euler method is protected longer by its inherently larger truncation error. Because its $\mathcal{O}(\Delta t)$ error decreases much more slowly, it acts as a mask that hides the underlying round-off accumulation. It is only at the highest resolution ($\Delta t = 10^{-4}$), where the massive step count maximizes the round-off accumulation, that the discretization error becomes small enough to let the machine noise dominate, forcing the total error to finally rebound and complete its own V-shape.

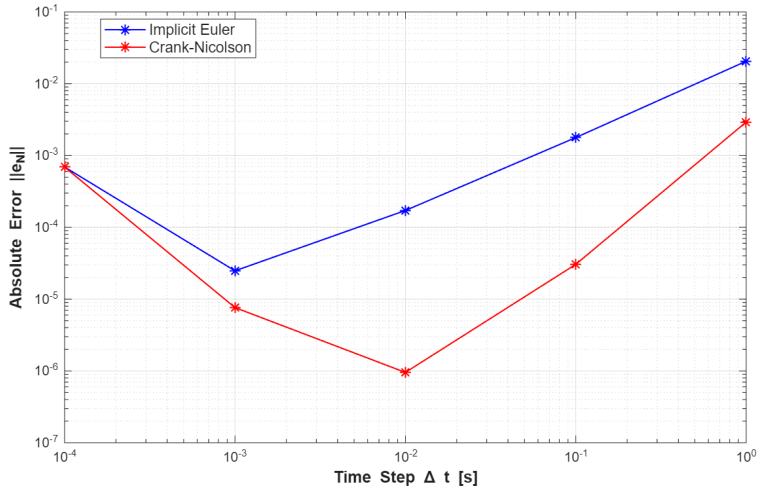

Figure 3: Absolute error at $T = 10$ vs Δt in single precision (log-log scale).

4 Conclusions

The numerical experiments conducted in this study successfully highlight the fundamental characteristics of implicit time-integration schemes applied to non-linear ODEs, as well as the intrinsic physical limitations of machine arithmetic.

First, the time evolution analysis (Task 1) demonstrated the superiority of the second-order Crank-Nicolson method over the first-order Implicit Euler scheme, successfully minimizing the discretization error even on exceptionally coarse temporal grids. The convergence analysis in double precision (Task 2) rigorously validated these formal orders of accuracy, yielding the theoretically expected linear and quadratic error reductions. Furthermore, the implementation of the Newton-Raphson algorithm proved highly effective and robust in handling the non-linear algebraic equations generated by the implicit nature of both schemes.

Finally, the comparison between single and double precision (Tasks 3 and 4) provided a clear visualization of the fundamental trade-off between truncation error and round-off error. The emergence of V-shaped error curves in single precision confirmed that indefinitely refining the time step does not continuously improve accuracy. Instead, when the discretization error drops below the non-linear solver's tolerance and the limits of the hardware precision, the massive accumulation of round-off noise actively degrades the global solution. Ultimately, this underscores the absolute necessity of utilizing double precision (`real64`) in scientific computing whenever highly refined grids or high-order accurate schemes are deployed.